

# AI Architecture Builder

Software Design Document • Version 1.0 • July 2026

---

## Project Overview

The AI Architecture Builder is an enterprise platform that uses a multi-agent AI system to automatically generate complete cloud architectures from requirement documents. Users upload project requirements, and the system produces system diagrams, database schemas, API designs, Terraform scripts, security validations, and cost estimates — all without manual architecture work.

## Objectives

- Automate end-to-end cloud architecture generation from requirement documents.
- Use 16 specialized AI agents that collaborate to produce production-ready output.
- Generate deployable Terraform code, database schemas, and API specifications.
- Provide AI-powered architecture review with quality scoring.
- Support interactive refinement through AI chat.

## Goals

- **Business:** Reduce architecture design time from weeks to hours. Standardize architecture practices across teams.
- **Technical:** Scalable multi-agent orchestration. Cloud-agnostic (AWS, Azure, GCP). Extensible agent framework.
- **AI:** Context-aware decisions using RAG. Explainable recommendations. Reliable validation with low false positives.

## Who Uses It

- **Architects** — accelerate design and validate decisions.
  - **Dev Teams** — generate architectures without waiting for architects.
  - **DevOps** — receive production-ready Terraform and pipelines.
  - **Managers** — review architectures with AI quality scores.
- 

## Phase 1 — Project Initialization & Authentication

Set up the monorepo project structure, configure the development environment, implement user authentication with JWT, create the database schema with migrations, and build the basic frontend shell with login, registration, and an empty dashboard. This is the foundation — nothing else works without auth and database.

## What to do

- Initialize monorepo with frontend (React + Vite + TypeScript) and backend (Python + FastAPI) packages.
- Set up SQLite/PostgreSQL database with migration tool (Alembic).
- Implement user registration, login, logout, password reset.
- JWT access tokens (15 min expiry) + refresh tokens (7 day) with rotation.
- Build frontend shell: login page, register page, sidebar + header layout, empty dashboard.
- Set up audit logging for all auth events.
- Configure ESLint, Prettier, Pytest, CI/CD pipeline.

## Folder Structure

```

ai-architecture-agent/
├── frontend/
│   ├── src/
│   │   ├── components/           # Reusable UI components
│   │   ├── common/              # Button, Input, Modal, Loader
│   │   ├── layout/              # Sidebar, Header, AppLayout
│   │   ├── pages/
│   │   │   ├── auth/            # LoginPage, RegisterPage, ForgotPassword
│   │   │   ├── dashboard/       # DashboardPage
│   │   │   ├── services/        # API client (axios instance, auth service)
│   │   │   ├── store/           # State management (Zustand or Redux)
│   │   │   ├── hooks/           # useAuth, useApi custom hooks
│   │   │   ├── utils/           # Helpers, validators, formatters
│   │   │   ├── types/           # TypeScript interfaces
│   │   │   ├── styles/          # Global CSS, design tokens
│   │   │   ├── routes/          # Route config with protected routes
│   │   │   ├── App.tsx          # Root component
│   │   │   └── main.tsx         # Entry point
│   │   ├── package.json
│   │   └── vite.config.ts
│   ├── backend/
│   │   ├── src/
│   │   │   ├── config/          # DB, auth, app config files
│   │   │   ├── controllers/     # auth_controller.py
│   │   │   ├── services/        # auth_service.py, audit_service.py
│   │   │   ├── models/         # user_model.py, organization_model.py
│   │   │   ├── middleware/      # auth_middleware.py, error_handler.py
│   │   │   ├── routes/         # auth_routes.py
│   │   │   ├── migrations/     # alembic migration files
│   │   │   ├── utils/          # jwt_helper.py, password.py, logger.py
│   │   │   ├── main.py          # FastAPI app entry point
│   │   │   └── requirements.txt
│   │   ├── docker-compose.yml   # PostgreSQL + Redis for local dev
│   │   ├── .github/workflows/   # CI/CD pipelines
│   │   └── README.md            # Project readme

```

## Phase 2 — Backend API Development

Build all backend REST APIs for project management, document upload and processing, architecture generation triggers, version management, review handling, deployment management, and chat. Each API module has a controller (handles request/response), a service (business logic), and a model (database). WebSocket support for real-time progress updates.

### What to do

- Build Project CRUD APIs — create, list, get, update, archive projects.
- Build Document Upload API — multipart upload, validate format (PDF/DOCX/TXT/MD), store in S3, trigger processing.

- Build Architecture APIs — trigger generation, get current architecture, get database schema, get API specs.
- Build Version APIs — list versions, get version, compare two versions, rollback.
- Build Review APIs — request review, list reviews, get review details, respond to findings.
- Build Deployment APIs — initiate, approve, list, rollback deployments.
- Build Chat APIs — create conversation, send message, get message history.
- Build Cloud Mapping and Terraform APIs — get mappings, get/download/validate Terraform.
- Set up WebSocket server for real-time generation progress and deployment status.
- Implement RBAC middleware (Viewer, Editor, Approver, Admin roles).
- Add rate limiting, input validation, and error handling middleware.

## API Reference

| Endpoint                                           | Purpose                       |
|----------------------------------------------------|-------------------------------|
| POST /api/auth/register                            | Create user account           |
| POST /api/auth/login                               | Login, returns JWT tokens     |
| POST /api/auth/refresh                             | Refresh access token          |
| POST /api/auth/logout                              | Invalidate session            |
| POST /api/projects                                 | Create new project            |
| GET /api/projects                                  | List user projects            |
| GET /api/projects/:id                              | Get project details           |
| PUT /api/projects/:id                              | Update project settings       |
| DELETE /api/projects/:id                           | Archive project               |
| POST /api/projects/:id/documents                   | Upload requirement document   |
| GET /api/projects/:id/documents                    | List project documents        |
| POST /api/projects/:id/generate                    | Start architecture generation |
| GET /api/projects/:id/architecture                 | Get current architecture      |
| GET /api/projects/:id/architecture/database        | Get database schema           |
| GET /api/projects/:id/architecture/apis            | Get API specification         |
| GET /api/projects/:id/cloud-mapping                | Get cloud service mapping     |
| GET /api/projects/:id/terraform                    | Get Terraform code            |
| GET /api/projects/:id/terraform/download           | Download Terraform as zip     |
| POST /api/projects/:id/terraform/validate          | Validate Terraform syntax     |
| POST /api/projects/:id/reviews                     | Request AI review             |
| GET /api/projects/:id/reviews/:rId                 | Get review with findings      |
| PUT /api/projects/:id/reviews/:rId/findings/:fId   | Accept/reject finding         |
| GET /api/projects/:id/versions                     | List architecture versions    |
| GET /api/projects/:id/versions/compare             | Compare two versions          |
| POST /api/projects/:id/versions/:vId/rollback      | Rollback to version           |
| POST /api/projects/:id/deployments                 | Initiate deployment           |
| POST /api/projects/:id/deployments/:dId/approve    | Approve deployment            |
| POST /api/projects/:id/deployments/:dId/rollback   | Rollback deployment           |
| POST /api/projects/:id/conversations               | Start chat conversation       |
| POST /api/projects/:id/conversations/:cId/messages | Send chat message             |

| Endpoint                                          | Purpose                                                                  |
|---------------------------------------------------|--------------------------------------------------------------------------|
| GET /api/projects/:id/conversations/:cld/messages | Get chat history                                                         |
| GET /api/projects/:id/diagrams/:type              | Get diagram (system-context, component, deployment, sequence, data-flow) |
| GET /api/projects/:id/workflow-status             | Get generation progress                                                  |

## Backend Folder Structure (Phase 2 additions)

```

backend/src/
├── controllers/
│   ├── auth_controller.py      # Login, register, token refresh
│   ├── project_controller.py   # Project CRUD operations
│   ├── document_controller.py  # File upload & processing
│   ├── architecture_controller.py # Architecture generation & retrieval
│   ├── terraform_controller.py # Terraform code & validation
│   ├── review_controller.py    # AI review & findings
│   ├── version_controller.py   # Version history & comparison
│   ├── deployment_controller.py # Deploy, approve, rollback
│   ├── chat_controller.py      # Conversations & messages
│   ├── diagram_controller.py   # Diagram generation & export
│   └── cloud_controller.py     # Cloud service catalog
├── services/
│   ├── auth_service.py         # JWT, password hashing, sessions
│   ├── project_service.py      # Project business logic
│   ├── document_service.py     # File parsing, text extraction
│   ├── architecture_service.py # Architecture storage & manipulation
│   ├── terraform_service.py    # Terraform code management
│   ├── review_service.py       # Review triggers & findings tracking
│   ├── version_service.py      # Snapshots, diffs, rollback
│   ├── deployment_service.py   # Terraform execution, status
│   ├── chat_service.py         # Message routing to agents
│   ├── notification_service.py # WebSocket & email notifications
│   └── audit_service.py        # Immutable audit log recording
├── models/
│   ├── user_model.py           # Users table
│   ├── organization_model.py    # Organizations table
│   ├── project_model.py        # Projects table
│   ├── document_model.py       # Uploaded documents table
│   ├── requirement_model.py     # Extracted requirements table
│   ├── architecture_version_model.py # Version snapshots table
│   ├── review_model.py         # Review results table
│   ├── review_finding_model.py # Individual findings table
│   ├── deployment_model.py     # Deployment records table
│   ├── conversation_model.py   # Chat conversations table
│   ├── message_model.py        # Chat messages table
│   ├── knowledge_model.py      # RAG knowledge base table
│   ├── agent_execution_model.py # Agent run logs table
│   └── audit_log_model.py      # Audit trail table
└── middleware/

```

```

■ ■■■ auth_middleware.py          # JWT verification
■ ■■■ rbac_middleware.py          # Role-based access control
■ ■■■ validation_middleware.py    # Request body validation
■ ■■■ rate_limit_middleware.py    # API rate limiting
■ ■■■ error_handler_middleware.py # Global error handler
■■■ routes/                      # Route files mapping URLs to controllers
■■■ migrations/                  # Alembic migration scripts folder
■■■ websocket/                   # WebSocket event handlers
■■■ utils/                       # JWT helpers, logger, S3 client

```

## Phase 3 — AI Agents

Build 16 specialized AI agents that collaborate through an Orchestrator. Each agent handles one domain — document processing, requirements, knowledge retrieval, architecture design, database, APIs, cloud mapping, Terraform, security, cost, review, validation, documentation, versioning, and deployment. Agents share context through a common workspace and communicate via the Orchestrator which manages execution order, dependencies, and error recovery.

### What to do

- Build a base agent class with common LLM calling logic, prompt management, and error handling.
- Build the Orchestrator Agent that manages the full workflow pipeline and handles retries/failures.
- Implement a shared workspace (context object) that accumulates all agent outputs.
- Build each of the 16 agents listed below with their specific prompts and processing logic.
- Set up vector database (Pinecone/Weaviate) for RAG knowledge storage.
- Write prompt templates for each agent stored in a prompts/ directory.
- Build agent execution logging for monitoring and debugging.

### Agents

| Agent               | Responsibility                                                                   |
|---------------------|----------------------------------------------------------------------------------|
| Orchestrator        | Manages workflow order, invokes agents, handles errors and retries.              |
| Document Processing | Parses uploaded docs, extracts structured text, identifies sections/tables.      |
| Requirement         | Extracts and categorizes requirements (functional, non-functional, constraints). |
| RAG                 | Retrieves relevant enterprise knowledge from vector DB using similarity search.  |
| Architecture        | Generates high-level system design — components, relationships, patterns.        |
| Database            | Designs schemas, tables, relationships, indexes, selects DB technology.          |
| API                 | Designs REST endpoints, request/response schemas, produces OpenAPI spec.         |
| Cloud Mapping       | Maps components to cloud services (AWS/Azure/GCP), estimates costs.              |
| Terraform           | Generates production-ready Terraform modules from cloud mappings.                |
| Security            | Validates architecture against security best practices and compliance rules.     |
| Cost Optimization   | Estimates cloud costs, identifies savings, suggests right-sizing.                |
| Architecture Review | Scores architecture on completeness, consistency, security, scalability.         |
| Validation          | Checks Terraform syntax, schema validity, cross-reference consistency.           |
| Documentation       | Generates architecture docs, API docs, deployment guides, ADRs.                  |
| Versioning          | Creates immutable snapshots, generates change diffs between versions.            |
| Deployment          | Executes Terraform init/plan/apply, monitors deployment, handles rollback.       |

## Agent Execution Flow

1. User uploads document  
↓
2. Document Processing Agent → structures the content  
↓
3. Requirement Agent → extracts categorized requirements  
↓
4. RAG Agent → retrieves enterprise knowledge  
↓
5. Architecture Agent → generates system design  
↓
6. Database Agent → creates schema | API Agent → designs endpoints (parallel)  
↓
7. Cloud Mapping Agent → selects cloud services, estimates costs  
↓
8. Terraform Agent → generates infrastructure code  
↓
9. Security Agent → validates security | Validation Agent → checks correctness (parallel)  
↓
10. Cost Optimization Agent → analyzes pricing  
↓
11. Architecture Review Agent → scores and produces findings  
↓
12. Documentation Agent → generates docs  
↓
13. Versioning Agent → creates version snapshot  
↓
14. User receives complete architecture

## Agents Folder Structure

```
agents/  
  src/  
    common/  
      base_agent.py      # Base class – LLM calls, retries, logging  
      context.py         # Shared workspace between agents  
      llm_client.py      # LLM API client (OpenAI/Groq/Anthropic)  
      prompt_manager.py  # Loads and fills prompt templates  
    orchestrator/  
      orchestrator_agent.py # Workflow manager, agent sequencing  
      workflow_manager.py  # Execution order, dependency graph  
      agent_registry.py   # Registers all available agents  
    document_processing/  
      document_agent.py   # Text extraction, structure parsing  
    requirement/  
      requirement_agent.py # Requirement identification & categorization  
    rag/  
      rag_agent.py        # Query formulation, knowledge retrieval  
      embeddings_service.py # Chunk text and generate embeddings  
      vector_store.py     # Pinecone client  
    architecture/  
      architecture_agent.py # System design, component selection
```

```

■ ■■■ database/
■ ■ ■■■ database_agent.py      # Schema design, tech selection
■ ■■■ api/
■ ■ ■■■ api_agent.py           # Endpoint design, OpenAPI generation
■ ■■■ cloud_mapping/
■ ■ ■■■ cloud_mapping_agent.py # Service selection, cost estimation
■ ■ ■■■ providers/
■ ■ ■■■ aws_catalog.py         # AWS service catalog & pricing
■ ■ ■■■ azure_catalog.py       # Azure service catalog & pricing
■ ■ ■■■ gcp_catalog.py         # GCP service catalog & pricing
■ ■■■ terraform/
■ ■ ■■■ terraform_agent.py     # Terraform code generation
■ ■ ■■■ generators/
■ ■ ■■■ aws/                   # AWS-specific Terraform modules
■ ■ ■■■ azure/                 # Azure-specific Terraform modules
■ ■ ■■■ gcp/                   # GCP-specific Terraform modules
■ ■■■ security/
■ ■ ■■■ security_agent.py      # Security validation & compliance
■ ■ ■■■ policies/              # Compliance rules (SOC2, HIPAA, GDPR)
■ ■■■ cost_optimization/
■ ■ ■■■ cost_agent.py          # Cost analysis & optimization
■ ■■■ review/
■ ■ ■■■ review_agent.py        # Quality scoring & recommendations
■ ■■■ validation/
■ ■ ■■■ validation_agent.py    # Syntax & consistency checks
■ ■■■ documentation/
■ ■ ■■■ documentation_agent.py # Doc generation
■ ■ ■■■ templates/             # Documentation templates
■ ■■■ versioning/
■ ■ ■■■ versioning_agent.py    # Snapshot creation
■ ■ ■■■ diff_engine.py         # Version comparison engine
■ ■■■ deployment/
■ ■ ■■■ deployment_agent.py    # Terraform execution & monitoring
■■■ prompts/                   # All prompt templates per agent
■■■ tests/                     # Agent unit tests
■■■ requirements.txt

```

---

**End of Document**